

Layer Collection

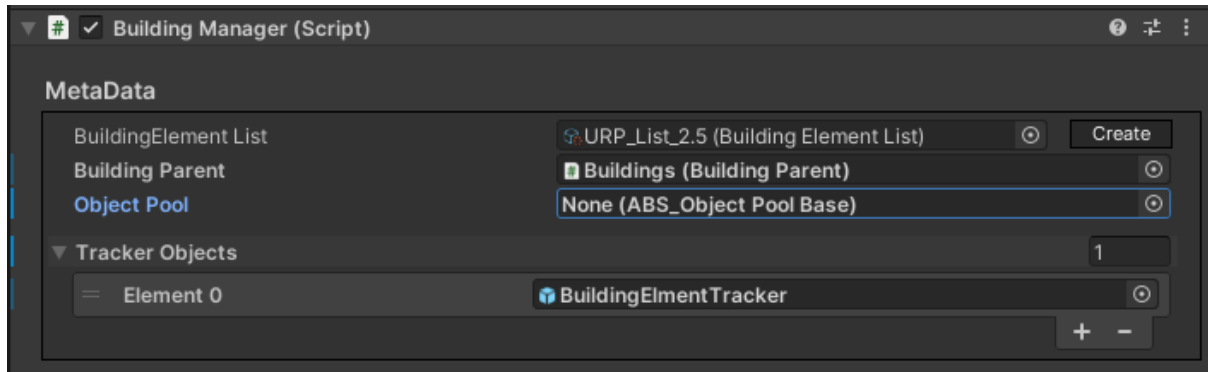
The LayerCollection is a Scriptable object holding the necessary layers for the Advanced Building Systems scripts.



- Layer Of Building Element
 - Only 1 layer can be set to ensure that the system works properly!
 - This layer will be used for the BuildingElements. Make sure your BuildingElements are on this layer.
- Layer Of Player
 - Only 1 layer can be set to ensure that the system works properly!
 - It is used to identify the Player.
 - It is used for blocking the elements during the build process if they are colliding with the player.
 - Also it is used for the [PreBuiltBuildingElement](#) feature.
- Layer Of Ground
 - It is used to identify the ground.
 - For example during the align to ground kind of features and underground check validation.
- Blocking Layer
 - Work similar to the Layer Of the Player but it is only used for blocking the elements if they are colliding with them during the build process.
- Raycast hit Layers
 - It is used for the raycast. What can be hit by the raycast.
 - Usually it is the mix of the Ground and the BuildingElement Layers.

ObjectPool/Buffer

The Advanced BuildingSystem can support custom object pools that can be provided to the BuildingManager on its UI. The developer can provide an object pool inherited from the ABS_ObjectPoolBase class. This is not a mandatory property so if it is null then the BuildingManager will instantiate and destroy the BuildingElement but if it is provided the manager will use it to increase the performance.



The ABS_ObjectPoolBase is an abstract class of 3 functions that should be implemented.

```
namespace REST.AdvancedBuildSystem.src
{
    public abstract class ABS_ObjectPoolBase : MonoBehaviour
    {
        public abstract BuildingElement Get(BuildingElement p_Element);
        public abstract void GiveBack(BuildingElement p_Element);
        public abstract void Release(BuildingElement p_Element);
    }
}
```

Get

When the Get function is called by the manager it will provide an element prefab as a parameter. The function should return a reference for an instance of the provided BuildingElement.

The return value can not be null!

The BuildingManager will set the necessary properties of the element like parent, rotation, position, layer etc.

GiveBack

When the GiveBack function is called the manager will provide a reference for an instance of a BuildingElement that can be put back to the pool.

The parent and the set properties are still in that state as the Get function set earlier. So at least the parent object should be reset by the GiveBack function and disable the object for preventing any building logic issue.

Release

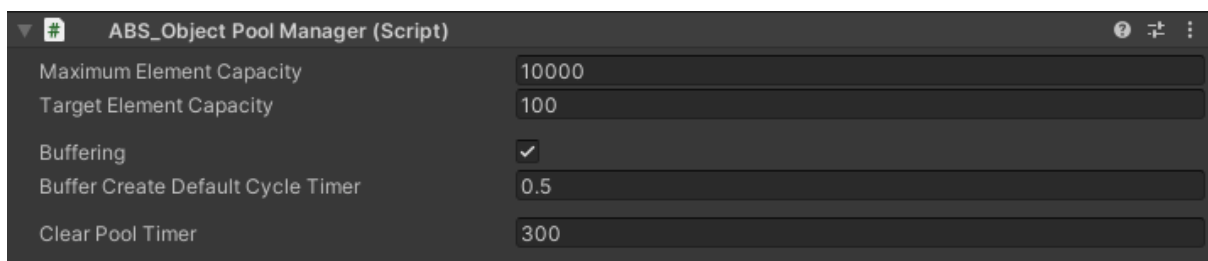
When the Release function is called the manager will provide a reference for an instance of a BuildingElement that has been placed to the world.

The pool should release its reference to that object because it will never be given back with the GiveBack function so it will just flood the pool's reference counter.

ABS_ObjectPoolManager

The Advanced Building System has its own implementation for the object pool that can be used as well. The ABS_ObjectPoolManager is inherited from the ABS_ObjectPoolBase and implements its required functions so no more action is required than just create an instance of the ABS_ObjectPoolManager and provide it to the BuildingManager on its UI.

This object pool included a buffering logic and a clear logic as well for performance and memory saving purposes.



First of all the manager itself is not the actual object pool, it just creates a pool for every different BuildingElement and forwards the function calls of the ABS_ObjectPoolBase to the right pool based on the BuildingElements. You can see the UI of the pools in action on the image later.

The buffering feature is about to create BuildingElements before it is ever used for performance purposes. A regular object pool when a huge amount of elements is requested and it hasn't any left in the pool then it will create the leftover. It can cause a huge amount of element instantiation in only one frame which can cause fps drop. To prevent this scenario the ABS_ObjectPoolManager's pools automatically create the BuildingElements in a cycle until it reaches the set count. The constant but slow GameObject creation can help to prevent fps drops. It works in such a way in the Update function of the pool the deltatime is constantly checked and when it's reached the timer it will create an element and put it into the pool.

Properties

Maximum Element Capacity

The maximum element counts what can be held by the pools separately. If the limit is reached the pool will return null for the Get function calls. The BuildingManager when it gets a null element it will instantiate the element itself in a regular way.

Target Element Capacity

The target element count of the buffering feature. The buffer cycle will create elements only when the inactive element count (elements in the pool) is smaller than this value. When it has been reached the buffer will stop creating the elements.

Buffering

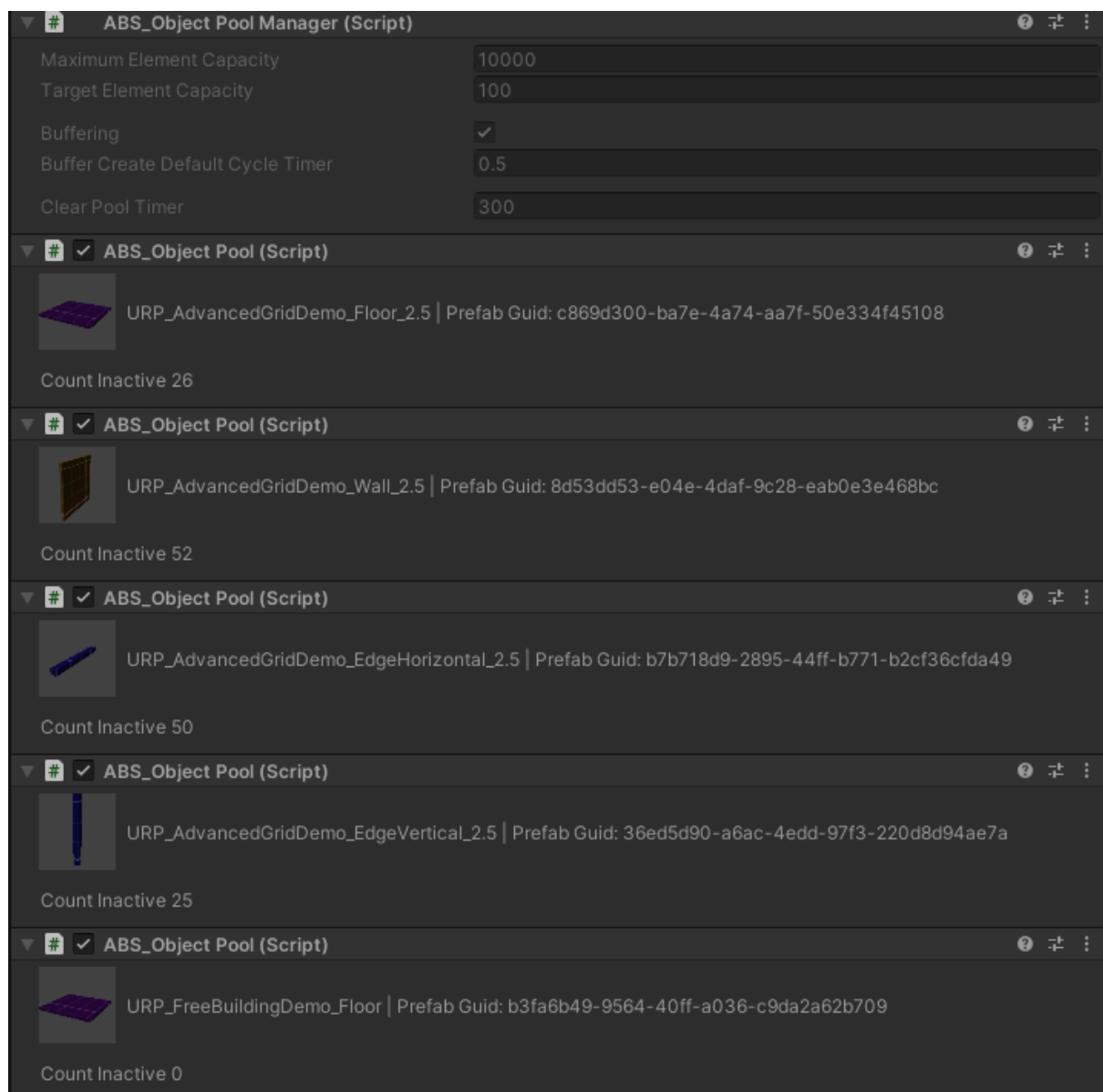
Boolean value which means that the buffering feature is enabled or not.

Buffer Create Default Cycle Timer

How much time should be waited between the element creation during the buffering cycle.

Clear Pool Timer

The pools will be cleared (destroyed with its inactive elements) if they didn't get any function calls until the set timer. The Get, GiveBack, Release function calls reset the timer.

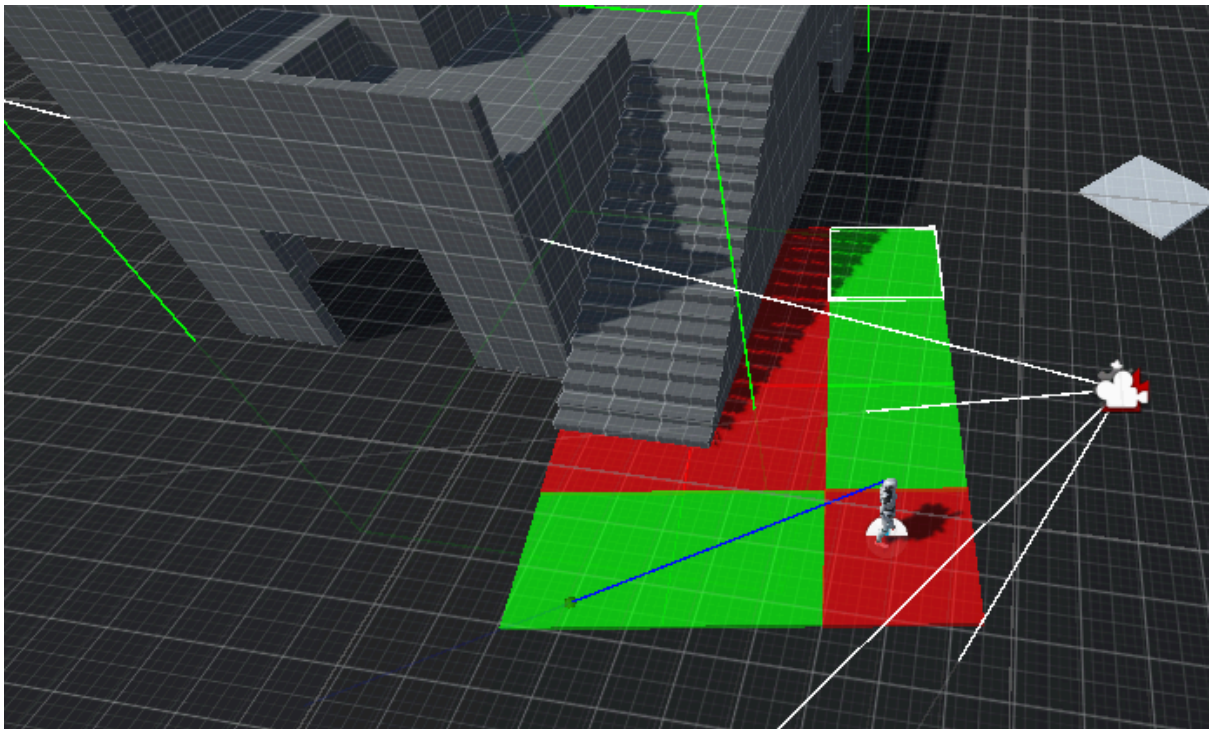


Building Area

The goal of the Building Area Feature is providing the possibility to the developers to create areas in the scene where they can manage the building of the elements. So basically it gives more possibility to the developers to create specific scenarios and manage the behavior of the Advanced Building System like:

- An element can be built near to something, for example an electrical element can be only built near to an electric source.
- Some elements can not be built near to something, for example a wooden element can not be built near the fire.

The main functionality of the BuildingAreas to allow or deny the building of the element.

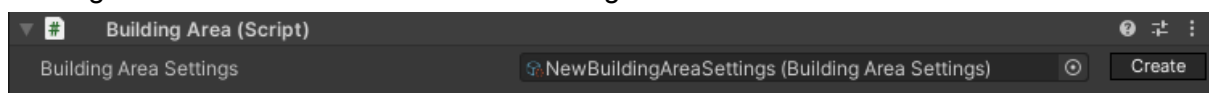


The Building Area created by the following components:

- Building Area (Script component)
- Building Area Settings (Scriptable Object)
- Building Area Ruleset (Scriptable Object)

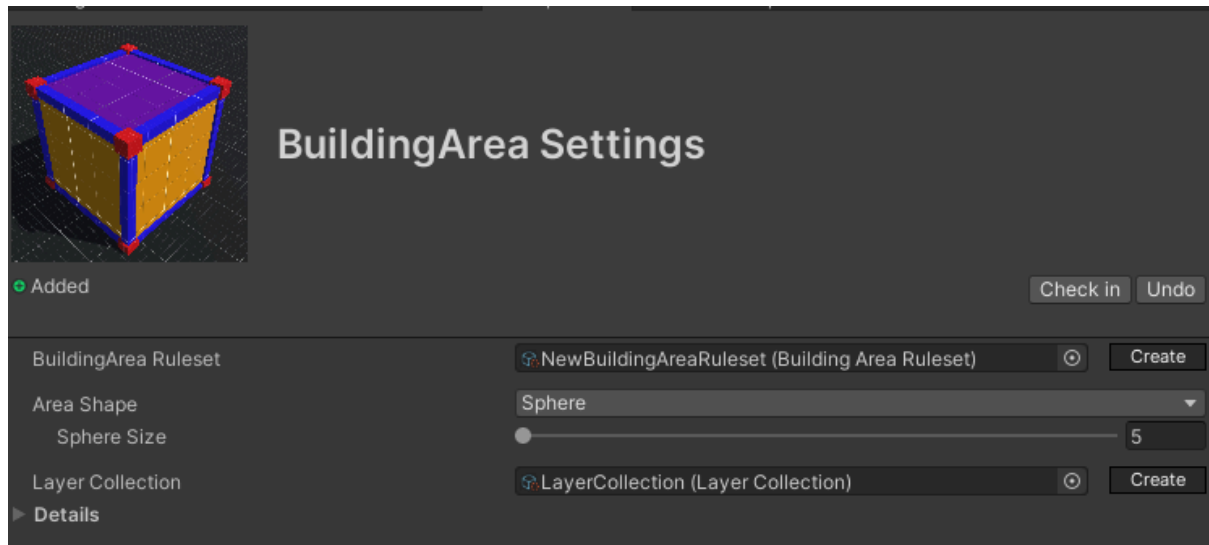
Building Area Component

The script component that is held by the GameObject's should have special area features. In the Awake function the Area will create a trigger collider and allow or deny the element's building inside that collider based on the settings.



Building Area Settings

The goal of the settings scriptable object is to give the possibility to reuse the same area in different situations so the developer doesn't have to set the same properties everytime.



BuildingArea Ruleset : The rules of the area. The allow and deny functionality based on that ruleset.

Area Shape : The shape of the area's collider. Currently 2 shapes are supported.

- Box
- Sphere

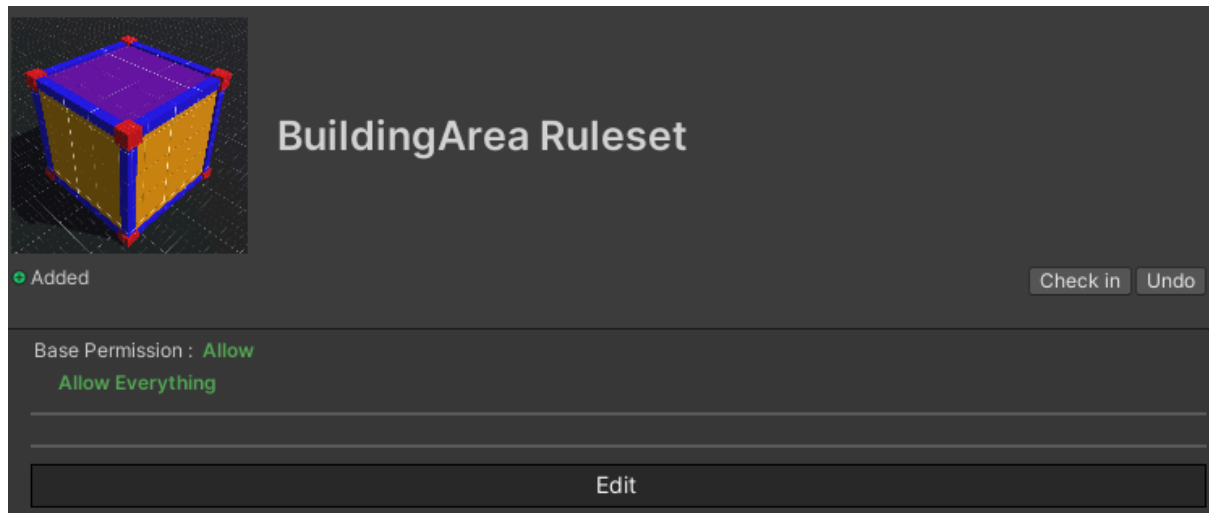
Sphere Size (If the shape is sphere) : The size/radius of the sphere collider created by the Building Area.

Box Size (If the shape is box) : The size of the box collider created by the Building Area in Vector3.

Layer Collection : The layer collection used by the Area.

Building Area Ruleset

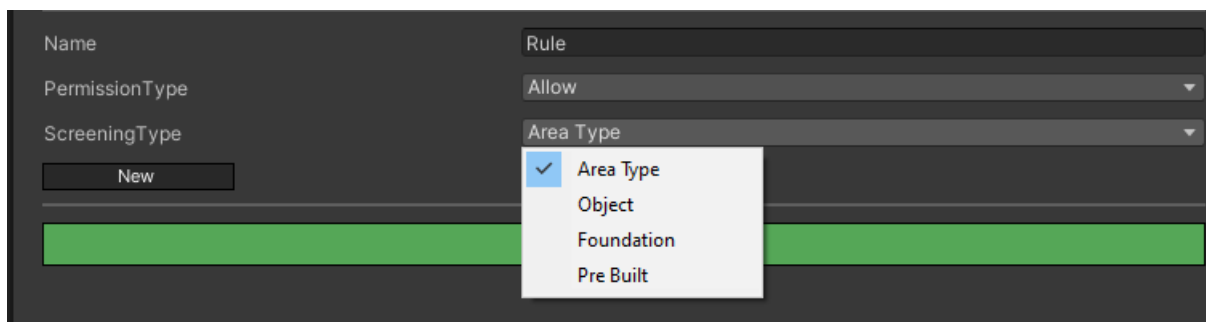
The goal of the ruleset scriptable object is to give the possibility to reuse the ruleset and give the possibility to use the same ruleset with different shaped areas on different areas. The goal is to give the maximum reusability and customizability to the Area feature of the Advanced Building System.



Base Permission: The base permission which will apply on every element what don't included by any of the rules. For example if the Base Permission is Deny and none of the rules said anything about the element (not allowed or denied that element) the building of that element will be blocked inside the area.

BuildingArea Rules have a PermissionType and a ScreeningType.

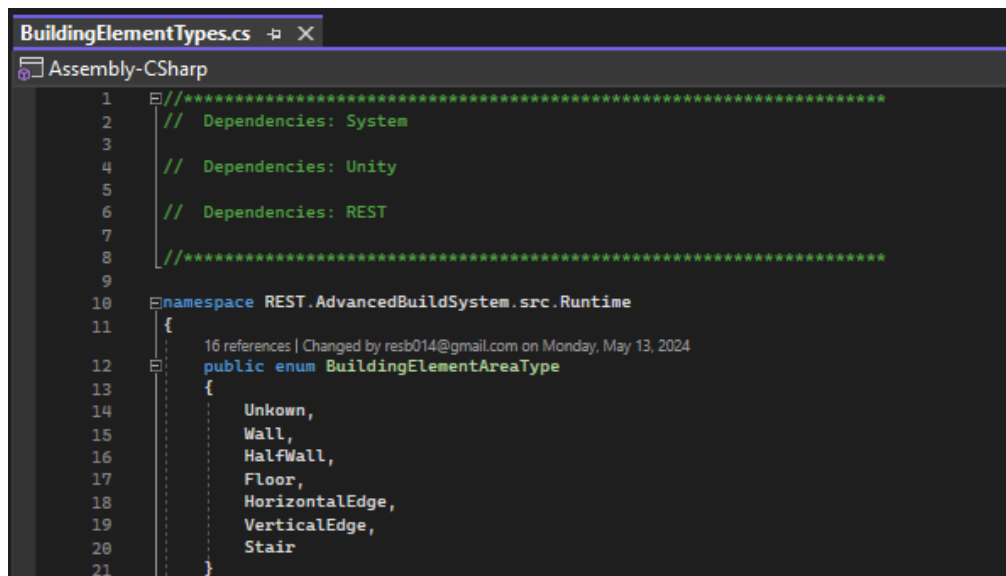
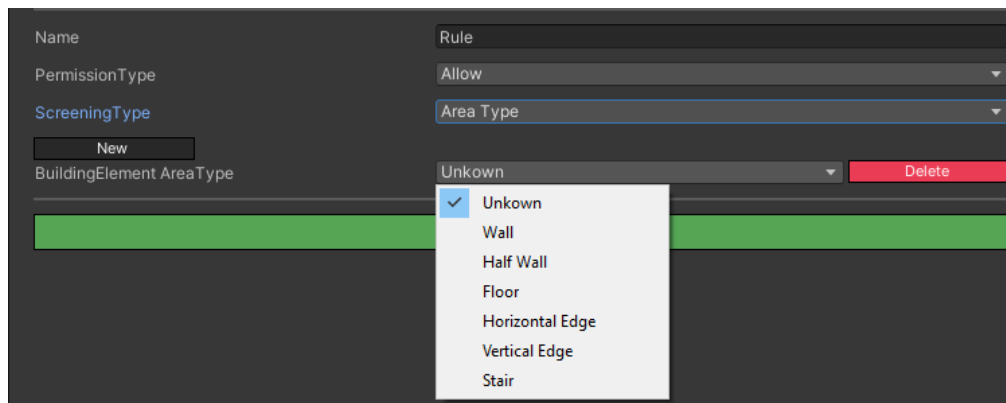
- Name
 - The main purposes are to identify the rule itself and make it easier to work with by the developer.
- Permission Type
 - Allow or Deny the elements that fit into the requirements of the rule.
- Screening Type
 - What should the Area check on the element to decide that it should allow or deny based on the Permission Type.



Area Type

The BuildingElement Area Type is an enum value that is defined in the BuildingElementTypes.cs. You the developer can change these enum values to create your logic. This is meant to identify the elements based on a fully independent type.

Note that every time when the AdvancedBuildingSystem asset is reimported this document can be overridden so ensure that you have a backup from your changes or just do not import this file again.



Building Element (Script)

BuildingElement MetaData: URP_AdvancedGridDemo_Wall_2.5 (Building) [Create]

Prefab Guid: 6dbc12f9-9e50-43a6-9f94-1b6fa08f [New Prefab Guid]

Fixed InstanceGuid: ☐

Final Element: None (Building Element) [v]

Snap To PreBuilt Final Element: ☐

Pre Built: ☒

Foundation: ☐

Area Type: Unkown [v]

Should Allowed By Area: ☐

Should Override: ☐

Indestructible: ☐

Highlight Properties

Highlight Strategy: Automatic Collection [v]

AdvancedGrid

URP_AdvancedGridDemo_Wall_2.5(Clone)

Snap to Grid (Advanced)

Object

If the Rule is set as an Object a set of BuildingElements can be defined. The defined elements will be denied or allowed by the Area.

Name: Rule

PermissionType: Allow [v]

ScreeningType: Object [v]

[New]

BuildingElement Object	URP_AdvancedGridDemo_Corner_2.5 (Building E) [v]	Delete
BuildingElement Object	URP_AdvancedGridDemo_Floor_2.5 (Building Ele) [v]	Delete

[Save]

Foundation

It simply denies or allows the building of the elements that are signed as Foundation.

Name	Rule
PermissionType	Allow
ScreeningType	Foundation
Save	

PreBuilt

Just like the Foundation logic denies or allows the building of the elements that are signed as [PreBuilt](#).

Name	Rule
PermissionType	Allow
ScreeningType	Pre Built
Save	

The Relationship of the Rules.

In the Editor view you can move the rules Up and Down or Top and Bottom.

It is important because the order of the checks are from top to bottom.

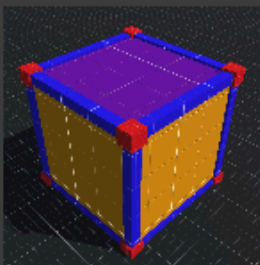
The area's behavior is that:

- If the rule is related to the element and allows it the element will be allowed.
- If the rule is related to the element and Denies it the element will be denied.
- If the rule is not related to the element then the next rule will be checked.

So if the rule is related then the element will be allowed or denied immediately when the rule says any of the two options. For example if the first rule allows the element and the second one denies it the element will be allowed.

Again, the order of the check is from top to bottom.

If all of the rules are unrelated to the element then the base permission will be applied.



BuildingArea Ruleset

Checkout

Base Permission : **Deny** Allow
Deny Everything

▼ Rule

Top Up Down Bottom

Edit Duplicate Delete

PermissionType : **Allow**

CollectionType : AreaType

(1) Wall

▼ Rule

Top Up Down Bottom

Edit Duplicate Delete

PermissionType : **Deny**

CollectionType : Object

0 :  URP_AdvancedGridDemo_Corner_2.5 | cc116ff3-a3a1-47d8-9014-4abbf0beed56

1 :  URP_AdvancedGridDemo_Floor_2.5 | 91ac1293-08c9-4c58-b080-19a8f9108bce

▼ Rule

Top Up Down Bottom

Edit Duplicate Delete

PermissionType : **Allow**

CollectionType : Foundation

▼ Rule

Top Up Down Bottom

Edit Duplicate Delete

PermissionType : **Allow**

CollectionType : PreBuilt

New Entity

Save